

4.8 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝

목차

1. 베이지안 최적화 개요

1. GridSearchCV의 한계
2. 베이지안 최적화의 원리
3. HyperOpt 소개와 설치

2. HyperOpt 기본 사용법

1. 검색 공간(Search Space) 설정
2. 목적 함수 생성
3. fmin()으로 최적값 탐색
4. 시도 횟수를 늘려 재탐색
5. Trials 객체 분석
6. DataFrame으로 결과 정리

3. HyperOpt를 이용한 XGBoost 하이퍼 파라미터 최적화

1. 데이터 준비
2. XGBoost 검색 공간 설정
3. 목적 함수 생성
4. fmin() 실행과 최적값 도출
5. 최적 파라미터로 재학습 및 평가

4. 핵심 요약 및 머신러닝 활용 포인트

1. 베이지안 최적화 개요

1.1 GridSearchCV의 한계

2.4장에서 배운 `GridSearchCV`는 **지정한 모든 조합을 전수 탐색**합니다. 직관적이고 확실하지만 결정적인 문제가 있습니다.

문제점	설명
조합 수의 폭발	파라미터가 늘어날수록 조합이 곱셈으로 증가 합니다. 5개 파라미터에 각 5개 값이면 3,125개 조합 이고, 여기에 5폴드 교차 검증이면 15,625번 학습 이 필요합니다.
이전 결과를 활용하지 못함	어떤 조합이 형편없었는지 알아도 다음 탐색에 전혀 반영하지 않고 무작정 모든 경우를 시도합니다.
이산적인 값만 탐색	학습률 같은 연속형 값 은 [0.01, 0.05, 0.1]처럼 미리 정해 둔 값만 시도할 수 있습니다.

1.2 베이지안 최적화의 원리

베이지안 최적화(Bayesian Optimization)는 **이전 탐색 결과를 학습**하여 다음에 시도할 값을 **지능적으로 선택**합니다. 미지의 목적 함수를 최소(또는 최대)로 만드는 최적해를 **가장 적은 시도**로 찾는 것이 목표입니다.

구성 요소	역할
대체 모델 (Surrogate Model)	지금까지의 입력값과 결과를 바탕으로 목적 함수의 형태를 추정 합니다. "이 근처는 좋을 것 같다"는 예측 지도를 그리는 셈입니다.
획득 함수 (Acquisition Function)	대체 모델의 추정을 바탕으로 다음에 시도할 후보를 추천 합니다. 좋아 보이는 영역을 더 파고들지(exploitation) , 아직 안 가 본 영역을 탐험할지(exploration) 의 균형을 맞춥니다.

■ 직관적인 비유

산에서 가장 낮은 골짜기를 찾는다고 생각해 보세요. **Grid Search**는 지도를 격자로 나눠 **모든 칸을 빠짐없이** 가 보는 방식입니다. 반면 **베이지안 최적화**는 몇 군데를 가 본 뒤 "**지형이 이렇게 생겼겠구나**"를 추정하고, **골짜기가 있을 법한 곳부터** 우선 탐색합니다. 훨씬 적은 발걸음으로 목적지에 도달할 수 있습니다.

1.3 HyperOpt 소개와 설치

```
#pip install hyperopt
```

줄	코드	상세 설명
1	<code>#pip install hyperopt</code>	주석 처리된 설치 명령 입니다. HyperOpt는 별도 패키지이므로 설치가 필요합니다. 주피터에서는 앞에 ! 를 붙여 <code>!pip install hyperopt</code> 로 실행할 수 있습니다.

```
import hyperopt

print(hyperopt.__version__)
```

▶ 실행 결과

0.3.0

■ 대표적인 베이지안 최적화 패키지

파이썬에는 **HyperOpt**, **Bayesian optimization**, **Optuna** 등이 있습니다. 이 중 HyperOpt가 가장 오래되고 널리 쓰이며, 최근에는 **Optuna**도 많이 사용됩니다. 원리는 모두 유사합니다.

2. HyperOpt 기본 사용법

실전에 들어가기 전에 **간단한 수식**으로 HyperOpt의 동작 원리를 익힙니다.

2.1 검색 공간(Search Space) 설정

```
from hyperopt import hp

# -10 ~ 10까지 1간격을 가지는 입력 변수 x와 -15 ~ 15까지 1간격으로 입력 변수 y 설정.
search_space = {'x': hp.quniform('x', -10, 10, 1), 'y': hp.quniform('y', -15, 15, 1) }
```

줄	코드	상세 설명
1	from hyperopt import hp	hp 는 검색 공간을 정의하는 함수들의 모음입니다.
4	search_space = {'x': hp.quniform('x', -10, 10, 1), ...}	검색 공간은 딕셔너리로 정의합니다. Key는 변수명, Value는 탐색 범위 지정 함수입니다. hp.quniform(label, low, high, q) 는 low부터 high까지 q 간격의 균등 분포에서 값을 추출합니다. 여기서는 x는 -10~10, y는 -15~15에서 1 간격(정수)으로 탐색합니다. 첫 인자 'x' 는 레이블로, 딕셔너리 Key와 동일하게 맞추는 것이 관례입니다.

■ 주요 검색 공간 함수

함수	설명	사용 예
hp.quniform(label, low, high, q)	q 간격의 이산 균등 분포	max_depth 같은 정수형
hp.uniform(label, low, high)	연속 균등 분포	learning_rate 같은 실수형
hp.randint(label, upper)	0부터 upper까지 정수	랜덤 시드 등
hp.choice(label, options)	리스트에서 하나 선택	criterion=['gini','entropy']
hp.loguniform(label, low, high)	로그 균등 분포	규제 계수처럼 스케일이 큰 값

2.2 목적 함수 생성

```
from hyperopt import STATUS_OK

# 목적 함수를 생성. 변수값과 변수 검색 공간을 가지는 딕셔너리를 인자로 받고, 특정 값을 반환
def objective_func(search_space):
    x = search_space['x']
    y = search_space['y']
    retval = x**2 - 20*y
```

```
return retval
```

줄	코드	상세 설명
4	<code>def objective_func(search_space):</code>	목적 함수는 HyperOpt가 최소화하려는 대상입니다. 인자로 검색 공간에서 추출된 값들의 딕셔너리를 받습니다.
5~6	<code>x = search_space['x'] / y = search_space['y']</code>	딕셔너리에서 개별 변수 값을 꺼냅니다.
7	<code>retval = x**2 - 20*y</code>	최소화할 수식입니다. x^2 는 $x=0$ 에서 최소(0)이고, $-20y$ 는 y 가 클수록 작아지므로 $y=15$ 에서 최소(-300)입니다. 따라서 이론적 최솟값은 $x=0, y=15$ 일 때 -300입니다. 정답을 미리 알고 있으니 HyperOpt가 잘 찾는지 검증할 수 있습니다.
9	<code>return retval</code>	계산 결과를 반환합니다. HyperOpt는 항상 이 반환값을 최소화하려 합니다.

⚠ 최대화 문제는 부호를 뒤집어야 한다

HyperOpt는 최소화만 수행합니다. 정확도처럼 클수록 좋은 지표를 최적화하려면 -1을 곱해 반환해야 합니다. 3장 실습에서 이 기법이 그대로 등장합니다.

2.3 fmin()으로 최적값 탐색

```
from hyperopt import fmin, tpe, Trials
import numpy as np

# 입력 결과값을 저장한 Trials 객체값 생성.
trial_val = Trials()

# 목적 함수의 최솟값을 반환하는 최적 입력 변수값을 5번의 입력값 시도(max_evals=5)로 찾아냄.
best_01 = fmin(fn=objective_func, space=search_space, algo=tpe.suggest, max_evals=5
               , trials=trial_val, rstate=np.random.default_rng(seed=0))
print('best:', best_01)
```

줄	코드	상세 설명
1	<code>from hyperopt import fmin, tpe, Trials</code>	fmin : 최적화 실행 함수 / tpe : 베이지안 최적화 알고리즘 / Trials : 시도 기록 저장 객체
5	<code>trial_val = Trials()</code>	모든 시도의 입력값과 결과를 저장할 객체입니다. 나중에 탐색 과정을 분석할 수 있습니다.
8~9	<code>fmin(fn=..., space=..., algo=tpe.suggest, max_evals=5, trials=..., rstate=...)</code>	최적화를 실행합니다. fn: 목적 함수 space: 검색 공간 algo=tpe.suggest: TPE(Tree-structured Parzen Estimator) 알고리즘. HyperOpt의 대표적인 베이지안 최적화 방식입니다. max_evals=5: 최대 5번만 시도 trials: 기록 저장 객체 rstate: 재현성을 위한 난수 시드
10	<code>print('best:', best_01)</code>	찾아낸 최적 입력값을 딕셔너리로 출력합니다.

▶ 실행 결과

```
100%|██████████| 5/5 [00:00<00:00, 1051.68trial/s, best loss: -224.0]
best: {'x': np.float64(-4.0), 'y': np.float64(12.0)}
```

■ 결과 해석

5번 시도로 $x=-4$, $y=12$ 를 찾았고 이때 목적 함수 값은 $(-4)^2 - 20 \times 12 = 16 - 240 = -224$ 입니다. 이론적 최솟값 $-300(x=0, y=15)$ 에는 못 미칩니다. 시도 횟수가 5회로 너무 적었기 때문입니다.

2.4 시도 횟수를 늘려 재탐색

```
trial_val = Trials()

# max_evals를 20회로 늘려서 재테스트
best_02 = fmin(fn=objective_func, space=search_space, algo=tpe.suggest, max_evals=20
               , trials=trial_val, rstate=np.random.default_rng(seed=0))
print('best:', best_02)
```

줄	코드	상세 설명
1	trial_val = Trials()	기록 객체를 새로 생성합니다. 재사용하면 이전 기록에 계속 누적되므로 새 실험에서는 새로 만듭니다.
4	max_evals=20	시도 횟수를 5 → 20회로 4배 늘렸습니다. 나머지 조건은 동일합니다.

▶ 실행 결과

```
100%|██████████| 20/20 [00:00<00:00, 1478.14trial/s, best loss: -296.0]
best: {'x': np.float64(2.0), 'y': np.float64(15.0)}
```

■ 결과 해석 - 시도 횟수의 중요성

max_evals	찾은 x, y	목적 함수 값	이론 최솟값과의 차이
5	$x=-4, y=12$	-224	76
20	$x=2, y=15$	-296	4

20회로 늘리자 y 는 최적값 15를 정확히 찾았고, x 도 0에 가까운 2를 찾아 -296이라는 거의 최적에 가까운 결과를 얻었습니다. 베이지안 최적화도 충분한 시도 횟수가 필요하지만, 전수 탐색($21 \times 31 = 651$ 가지)에 비하면 20회는 극히 적은 횟수입니다.

2.5 Trials 객체 분석

```
# fmin( )에 인자로 들어가는 Trials 객체의 result 속성에 파이썬 리스트로 목적 함수 반환값들이 저장됨
# 리스트 내부의 개별 원소는 {'loss': 함수 반환값, 'status': 반환 상태값} 와 같은 딕셔너리임.
print(trial_val.results)
```

줄	코드	상세 설명
3	trial_val.results	모든 시도의 결과값 리스트입니다. 각 원소는 <code>{ 'loss': 값, 'status': 'ok' }</code> 형태의 딕셔너리입니다. 순서는 시도 순서와 같습니다.

▶ 실행 결과

```
[{'loss': -64.0, 'status': 'ok'}, {'loss': -184.0, 'status': 'ok'}, {'loss': 56.0, 'status': 'ok'},
{'loss': -224.0, 'status': 'ok'}, {'loss': 61.0, 'status': 'ok'}, {'loss': -296.0, 'status': 'ok'},
{'loss': -40.0, 'status': 'ok'}, {'loss': 281.0, 'status': 'ok'}, {'loss': 64.0, 'status': 'ok'},
{'loss': 100.0, 'status': 'ok'}, {'loss': 60.0, 'status': 'ok'}, {'loss': -39.0, 'status': 'ok'},
{'loss': 1.0, 'status': 'ok'}, {'loss': -164.0, 'status': 'ok'}, {'loss': 21.0, 'status': 'ok'},
{'loss': -56.0, 'status': 'ok'}, {'loss': 284.0, 'status': 'ok'}, {'loss': 176.0, 'status': 'ok'},
{'loss': -171.0, 'status': 'ok'}, {'loss': 0.0, 'status': 'ok'}]
```

Trials 객체의 vals 속성에 {'입력변수명': 개별 수행 시마다 입력된 값 리스트} 형태로 저장됨.
print(trial_val.vals)

줄	코드	상세 설명
2	trial_val.vals	모든 시도의 입력값 입니다. {'x': [값들], 'y': [값들]} 형태로 변수별 리스트 로 저장됩니다. results 와 인덱스 가 서로 대응합니다.

▶ 실행 결과

```
{'x': [-6.0, -4.0, 4.0, -4.0, 9.0, 2.0, 10.0, -9.0, -8.0, -0.0,
      -0.0, 1.0, 9.0, 6.0, 9.0, 2.0, -2.0, -4.0, 7.0, -0.0],
'y': [5.0, 10.0, -2.0, 12.0, 1.0, 15.0, 7.0, -10.0, 0.0, -5.0,
      -3.0, 2.0, 4.0, 10.0, 3.0, 3.0, -14.0, -8.0, 11.0, -0.0]}
```

2.6 DataFrame으로 결과 정리

```
import pandas as pd

# results에서 loss 키값에 해당하는 밸류들을 추출하여 list로 생성.
losses = [loss_dict['loss'] for loss_dict in trial_val.results]

# DataFrame으로 생성.
result_df = pd.DataFrame({'x': trial_val.vals['x'], 'y': trial_val.vals['y'], 'losses': losses})
result_df
```

줄	코드	상세 설명
4	losses = [loss_dict['loss'] for loss_dict in trial_val.results]	리스트 컴프리헨션 으로 각 딕셔너리에서 'loss' 값만 뽑아 리스트로 만듭니다. results 는 딕셔너리 리스트라 바로 DataFrame에 넣기 불편하기 때문입니다.
7	result_df = pd.DataFrame({'x': ..., 'y': ..., 'losses': losses})	입력값과 결과를 하나의 표로 결합 합니다. 딕셔너리의 Key가 컬럼명이 됩니다. 이렇게 하면 탐색 과정을 한눈에 분석할 수 있습니다.

▶ 실행 결과

```
   x    y  losses
0 -6.0  5.0  -64.0
1 -4.0 10.0 -184.0
2  4.0 -2.0   56.0
3 -4.0 12.0 -224.0
4  9.0  1.0   61.0
5  2.0 15.0 -296.0 ← 최적
6 10.0  7.0  -40.0
7 -9.0 -10.0 281.0
8 -8.0  0.0   64.0
9 -0.0 -5.0  100.0
10 -0.0 -3.0   60.0
11  1.0  2.0  -39.0
12  9.0  4.0    1.0
```

```

13  6.0  10.0 -164.0
14  9.0   3.0  21.0
15  2.0   3.0 -56.0
16 -2.0 -14.0 284.0
17 -4.0  -8.0 176.0
18  7.0  11.0 -171.0
19 -0.0  -0.0   0.0

```

■ 결과 해석 - 탐색 과정 읽기

- **6번째 시도(인덱스 5)**에서 최적값 -296을 발견했습니다. 20번 중 **초반부에** 좋은 값을 찾은 것입니다.
- 이후에도 계속 탐색하는 이유는 **더 좋은 값이 있는지 확인**하기 위함입니다(exploration).
- 인덱스 7, 16처럼 **losses가 281, 284로 매우 나쁜** 시도도 있습니다. 이는 알고리즘이 **아직 탐색하지 않은 영역**을 확인하는 과정으로, 국소 최적에 빠지지 않기 위해 필요합니다.
- y값을 보면 **큰 값(10, 12, 15)에서 losses가 낮다**는 패턴을 알고리즘이 점차 학습해 가는 것을 볼 수 있습니다.

3. HyperOpt를 이용한 XGBoost 하이퍼 파라미터 최적화

3.1 데이터 준비

```

# 아래 코드는 이전에 수록된 코드라 책에는 실지 않았습니다.
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')

dataset = load_breast_cancer()

cancer_df = pd.DataFrame(data=dataset.data, columns=dataset.feature_names)
cancer_df['target'] = dataset.target
X_features = cancer_df.iloc[:, :-1]
y_label = cancer_df.iloc[:, -1]

```

줄	코드	상세 설명
9~14	dataset = load_breast_cancer() 등	4.6~4.7장과 동일한 유방암 데이터 를 준비합니다. 같은 데이터로 실습해야 튜닝 효과를 비교 할 수 있습니다.

```

# 전체 데이터 중 80%는 학습용 데이터, 20%는 테스트용 데이터 추출
X_train, X_test, y_train, y_test=train_test_split(X_features, y_label, test_size=0.2, random_state=156 )

# 앞에서 추출한 학습 데이터를 다시 학습과 검증 데이터로 분리
X_tr, X_val, y_tr, y_val= train_test_split(X_train, y_train, test_size=0.1, random_state=156 )

```

줄	코드	상세 설명
2	train_test_split(..., test_size=0.2)	학습 455건 / 테스트 114건으로 분리합니다.
5	X_tr, X_val, ... = train_test_split(X_train, y_train, test_size=0.1)	학습 409건 / 검증 46건으로 재분리합니다. 검증 세트는 마지막 조기 중단 에 사용됩니다.

3.2 XGBoost 검색 공간 설정

```
from hyperopt import hp

# max_depth는 5에서 20까지 1간격으로, min_child_weight는 1에서 2까지 1간격으로
# colsample_bytree는 0.5에서 1사이, learning_rate는 0.01에서 0.2 사이 정규 분포된 값으로 검색.
xgb_search_space = {'max_depth': hp.quniform('max_depth', 5, 20, 1),
                    'min_child_weight': hp.quniform('min_child_weight', 1, 2, 1),
                    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2),
                    'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 1),
                    }
```

줄	코드	상세 설명
5	'max_depth': hp.quniform('max_depth', 5, 20, 1)	트리 깊이를 5~20까지 1 간격 의 이산값으로 탐색합니다. 정수형 파라미터이므로 quniform 을 씁니다.
6	'min_child_weight': hp.quniform('min_child_weight', 1, 2, 1)	1과 2 두 값만 탐색합니다. 범위가 좁아 사실상 두 가지 선택지입니다.
7	'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)	학습률은 연속형 이므로 uniform 을 씁니다. 0.01~0.2 사이의 임의 실수 를 탐색할 수 있습니다. 이것이 GridSearchCV 대비 큰 장점 입니다 — 미리 값을 나열할 필요가 없습니다.
8	'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 1)	트리별 피쳐 샘플링 비율을 0.5~1 사이 실수로 탐색합니다.

■ GridSearchCV였다면

같은 범위를 GridSearchCV로 탐색하려면 **max_depth** 16개 × **min_child_weight** 2개 × **learning_rate** 예: 20개 × **colsample_bytree** 예: 10개 = **6,400개 조합**이 필요하며, 그마저도 연속값을 **이산적으로 근사**한 것입니다. HyperOpt는 이를 **50회 시도**로 처리합니다.

3.3 목적 함수 생성

```
from sklearn.model_selection import cross_val_score
from xgboost import XGBClassifier
from hyperopt import STATUS_OK

# fmin()에서 입력된 search_space 값으로 입력된 모든 값은 실수형임.
# XGBClassifier의 정수형 하이퍼 파라미터는 정수형 변환을 해줘야 함.
# 정확도는 높을수록 더 좋은 수치임. -1 * 정확도를 곱해서 큰 정확도 값일수록 최소가 되도록 변환
def objective_func(search_space):
    # 수행 시간 절약을 위해 n_estimators는 100으로 축소
    xgb_clf = XGBClassifier(n_estimators=100, max_depth=int(search_space['max_depth']),
                           min_child_weight=int(search_space['min_child_weight']),
                           learning_rate=search_space['learning_rate'],
                           colsample_bytree=search_space['colsample_bytree'],
                           eval_metric='logloss')
    accuracy = cross_val_score(xgb_clf, X_train, y_train, scoring='accuracy', cv=3)

    # accuracy는 cv=3 개수만큼 roc-auc 결과를 리스트로 가짐. 이를 평균해서 반환하되 -1을 곱함.
    return {'loss': -1 * np.mean(accuracy), 'status': STATUS_OK}
```

줄	코드	상세 설명
---	----	-------

10	XGBClassifier(n_estimators=100, ...)	목적 함수가 50번 호출 되고 각각 3폴드 교차 검증을 하므로 총 150번 학습합니다. 시간 절약을 위해 트리 수를 100으로 줄였습니다.
10~11	max_depth=int(search_space['max_depth'])	매우 중요한 처리입니다. HyperOpt는 <code>quniform</code> 으로 지정해도 모든 값을 실수(float)로 반환 합니다. <code>max_depth</code> 는 정수만 받으므로 <code>int()</code> 로 변환하지 않으면 오류가 발생합니다.
12~13	learning_rate / colsample_bytree	이 둘은 원래 실수형 이므로 변환 없이 그대로 전달합니다.
14	eval_metric='logloss'	평가 지표를 명시해 불필요한 경고를 방지합니다.
15	accuracy = cross_val_score(xgb_clf, X_train, y_train, scoring='accuracy', cv=3)	3폴드 교차 검증 으로 정확도를 측정합니다. 단일 분할보다 안정적인 성능 추정 이 가능해 튜닝의 신뢰도가 높아집니다. 반환값은 폴드별 점수 배열입니다.
18	return {'loss': -1 * np.mean(accuracy), 'status': STATUS_OK}	두 가지 핵심이 있습니다. ① -1 을 곱하는 이유: 정확도는 클수록 좋지만 HyperOpt는 최소화 만 하므로, 부호를 뒤집어 정확도가 높을수록 loss가 작아지게 만듭니다. ② 딕셔너리 반환 : 앞의 기본 예제는 숫자를 그대로 반환했지만, 실전에서는 <code>{'loss': 값, 'status': STATUS_OK}</code> 형태로 반환하는 것이 표준 방식 입니다.

3.4 fmin() 실행과 최적값 도출

```
from hyperopt import fmin, tpe, Trials

trial_val = Trials()
best = fmin(fn=objective_func,
           space=xgb_search_space,
           algo=tpe.suggest,
           max_evals=50, # 최대 반복 횟수를 지정합니다.
           trials=trial_val, rstate=np.random.default_rng(seed=9))
print('best:', best)
```

줄	코드	상세 설명
7	max_evals=50	50회 시도 합니다. 각 시도마다 3폴드 교차 검증이 수행되므로 총 150회 학습 이 이뤄집니다.
8	rstate=np.random.default_rng(seed=9)	재현성을 위한 시드입니다. 시드가 다르면 결과도 달라집니다.

▶ 실행 결과

```
best: {'colsample_bytree': np.float64(0.858999866296754),
      'learning_rate': np.float64(0.17459146963933464),
      'max_depth': np.float64(20.0),
      'min_child_weight': np.float64(2.0)}
```

```
print('colsample_bytree:{0}, learning_rate:{1}, max_depth:{2}, min_child_weight:{3}'.format(
    round(best['colsample_bytree'], 5), round(best['learning_rate'], 5),
    int(best['max_depth']), int(best['min_child_weight'])))
```

줄	코드	상세 설명
1~3	round(...) / int(...)	결과를 보기 좋게 정리 합니다. 실수형은 <code>round()</code> 로 소수점 5자리까지, 정수형 파라미터는 <code>int()</code> 로 변환합니다. best의 값들도 모두 실수형 이므로 이 변환이 필요합니다.

▶ 실행 결과

```
colsample_bytree:0.859, learning_rate:0.17459, max_depth:20, min_child_weight:2
```

■ 결과 해석

`learning_rate` 가 **0.17459**라는 값에 주목하세요. GridSearchCV였다면 [0.05, 0.1, 0.15, 0.2] 같이 **미리 정해 둔 값 중에서만** 골랐을 텐데, HyperOpt는 **연속 구간에서 임의의 실수**를 찾아냈습니다. 이것이 베이지안 최적화의 강점입니다.

`max_depth=20` 은 탐색 범위의 최댓값입니다. 이런 경우 **범위를 더 넓혀서 재탐색**해 볼 필요가 있다는 신호입니다.

3.5 최적 파라미터로 재학습 및 평가

```
xgb_wrapper = XGBClassifier(n_estimators=400,
                             learning_rate=round(best['learning_rate'], 5),
                             max_depth=int(best['max_depth']),
                             min_child_weight=int(best['min_child_weight']),
                             colsample_bytree=round(best['colsample_bytree'], 5)
                             )

evals = [(X_tr, y_tr), (X_val, y_val)]
xgb_wrapper.fit(X_tr, y_tr, early_stopping_rounds=50, eval_metric='logloss',
                eval_set=evals, verbose=True)

preds = xgb_wrapper.predict(X_test)
pred_proba = xgb_wrapper.predict_proba(X_test)[:, 1]

get_clf_eval(y_test, preds, pred_proba)
```

줄	코드	상세 설명
1	<code>XGBClassifier(n_estimators=400, ...)</code>	탐색 때는 100이었던 트리 수를 400으로 늘립니다 . 탐색은 빠르게, 최종 학습은 충실하게 하는 전형적인 전략입니다.
2~5	<code>learning_rate=round(best[...], 5)</code> 등	HyperOpt가 찾은 최적 파라미터를 그대로 적용 합니다. 정수형은 <code>int()</code> 변환을 잊지 않습니다.
8	<code>evals = [(X_tr, y_tr), (X_val, y_val)]</code>	조기 중단 기준으로 검증 세트를 사용 합니다. 테스트 세트를 쓰지 않는 올바른 방식 입니다.
9~10	<code>fit(..., early_stopping_rounds=50, eval_metric='logloss', ...)</code>	50회 조기 중단으로 학습합니다. 단, XGBoost 2.0부터 이 인자들은 생성자로 옮겨야 합니다(아래 경고 박스).
15	<code>get_clf_eval(y_test, preds, pred_proba)</code>	4.6장에서 정의한 평가 함수를 재사용 합니다.

⚠ XGBoost 2.0+ 필수 수정

`fit()` 의 `early_stopping_rounds`, `eval_metric` 은 제거되었습니다. 안정화 버전에서는 아래처럼 **생성자로 이동**해야 합니다.

```
xgb_wrapper = XGBClassifier(n_estimators=400,
                             learning_rate=round(best['learning_rate'], 5),
                             max_depth=int(best['max_depth']),
                             min_child_weight=int(best['min_child_weight']),
                             colsample_bytree=round(best['colsample_bytree'], 5),
                             early_stopping_rounds=50, eval_metric='logloss')
xgb_wrapper.fit(X_tr, y_tr, eval_set=[(X_tr, y_tr), (X_val, y_val)], verbose=True)
```

▶ 실행 결과 (수정된 코드 기준)

```
best_iteration: 138
```

```
오차 행렬
```

```
[[35  2]
```

```
 [ 3 74]]
```

```
정확도: 0.9561, 정밀도: 0.9737, 재현율: 0.9610,   F1: 0.9673, AUC:0.9944
```

■ 결과 해석 - 튜닝이 항상 성능을 올리지는 않는다

모델	정확도	AUC
4.6장 기본 XGBoost (네이티브, 조기중단)	0.9649	0.9965
4.6장 사이킷런 래퍼 (기본 파라미터)	0.9737	0.9951
4.8장 HyperOpt 튜닝	0.9561	0.9944

튜닝한 모델이 오히려 기본 파라미터보다 낮게 나왔습니다. 의외의 결과이지만 이유가 있습니다.

- 테스트 데이터가 114건으로 매우 적습니다. 1~2건 차이로 정확도가 1%p 가까이 움직이므로 이 정도 차이는 통계적으로 유의미하지 않습니다.
- HyperOpt는 학습 데이터의 교차 검증 정확도를 최적화했습니다. 그 목표에서는 성공했지만, 작은 테스트 세트에서의 성과 항상 일치하지는 않습니다.
- `max_depth=20` 은 상당히 깊은 트리로, 569건짜리 작은 데이터에는 과할 수 있습니다.

교훈: 하이퍼 파라미터 튜닝은 만능이 아닙니다. 데이터가 적을 때는 튜닝 효과보다 분할에 따른 변동이 더 클 수 있으며, 성능 비교는 충분한 데이터와 교차 검증으로 판단해야 합니다.

4. 핵심 요약 및 머신러닝 활용 포인트

4.1 GridSearchCV vs HyperOpt

항목	GridSearchCV	HyperOpt (베이지안)
탐색 방식	전수 탐색	이전 결과 기반 지능적 탐색
연속형 값	미리 나열한 값만	구간 내 임의 실수
시도 횟수	조합 수만큼 (기하급수적)	지정한 <code>max_evals</code>

파라미터 많을 때	사실상 불가능	효율적
구현 난이도	쉬움 (딕셔너리 하나)	목적 함수 직접 작성 필요
결과 재현	항상 동일	<code>rstate</code> 고정 필요

4.2 핵심 API 요약표

API / 속성	설명	주의 사항
<code>hp.quniform(label, low, high, q)</code>	q 간격 이산 균등 분포	반환값은 실수 → <code>int()</code> 변환 필요
<code>hp.uniform(label, low, high)</code>	연속 균등 분포	학습률 등 실수형 파라미터에 사용
<code>hp.choice(label, options)</code>	리스트에서 선택	반환값이 인덱스인 점 주의
<code>Trials()</code>	시도 기록 저장 객체	새 실험마다 새로 생성
<code>fmin(fn, space, algo, max_evals, trials, rstate)</code>	최적화 실행	항상 최소화 — 최대화는 -1 곱하기
<code>tpe.suggest</code>	TPE 베이지안 알고리즘	가장 일반적인 선택
<code>trials.results</code>	결과 딕셔너리 리스트	<code>[{'loss': 값, 'status': 'ok'}, ...]</code>
<code>trials.vals</code>	입력값 딕셔너리	<code>{'변수명': [값들]}</code>
<code>STATUS_OK</code>	정상 수행 상태값	딕셔너리 반환 시 함께 전달

4.3 자주 하는 실수와 대처

실수	증상	해결
정수 변환 누락	<code>max_depth</code> 에 실수 전달로 오류	<code>int(search_space['max_depth'])</code>
최대화 문제에 -1 미적용	가장 나쁜 파라미터를 찾음	<code>return -1 * 정확도</code>
Trials 객체 재사용	이전 실험 기록이 누적	실험마다 <code>Trials()</code> 새로 생성
<code>max_evals</code> 부족	최적값에 도달하지 못함	파라미터 수에 비례해 늘리기
최적값이 범위 경계	더 좋은 값을 놓쳤을 가능성	검색 범위를 넓혀 재탐색

4.4 머신러닝 활용 포인트

- **파라미터가 3개 이상이면 베이지안 최적화** : 1~2개라면 GridSearchCV가 간편하지만, 그 이상은 HyperOpt가 압도적으로 효율적입니다.
- **탐색은 가볍게, 최종 학습은 충실하게** : 탐색 중에는 `n_estimators` 를 줄이고, 최적값을 찾은 뒤 늘려서 재학습하는 전략이 표준입니다.
- **교차 검증을 목적 함수에 포함** : 단일 분할 점수로 튜닝하면 그 분할에 과적합됩니다. `cross_val_score` 를 쓰세요.
- **경계값이 나오면 범위 재조정** : 최적값이 탐색 범위의 끝(본 실습의 `max_depth=20`)이면 범위를 넓혀 다시 탐색해야 합니다.
- **튜닝 효과를 과신하지 말 것** : 데이터가 적으면 튜닝 효과보다 분할 변동이 더 큼니다. 기본 파라미터 성능과 반드시 비교하세요.
- **대안 검토** : 최근에는 Optuna가 더 직관적인 API와 시각화 기능으로 인기를 얻고 있습니다. 원리는 동일하므로 쉽게 전환할 수 있습니다.

⚠ 안정화 버전 참고

- **HyperOpt 0.3.0**: `rstate` 에 `np.random.RandomState(seed)` 대신 `np.random.default_rng(seed)` 를 사용해야 합니다. 구버전 코드는 오류가 발생합니다.
- **XGBoost 2.0+**: `fit( )` 의 `early_stopping_rounds`, `eval_metric` 이 제거되어 **생성자로 이동**했습니다.
- `fmin( )` 의 `best` 반환값은 **모든 값이 numpy 실수형**입니다. 모델에 넣기 전 반드시 타입을 확인하세요.
- 진행 표시줄을 끄려면 `fmin(..., show_progressbar=False)` 를 사용합니다.